# Chapter 6

# BOUNDARY MODELS

Decomposition models and constructive models both view solids as point sets, and seek representations for the point set either by discretizing it or by constructing it from simpler point sets. In contrast to these models, *boundary models* represent a solid indirectly through a representation of its bounding surface.

## 6.1 BASIC CONCEPTS

Historically, boundary models emerged from the polyhedral models used in computer graphics for representing objects and scenes for hidden line and surface removal. They can be viewed as "enhanced" graphical models that attempt to overcome the problems of graphical models by including a complete description of the bounding surfaces of the object.

Boundary models are based on the surface-oriented view to solid modeling represented in Section 3.5. That is, they represent a solid object by dividing its surface into a collection of *faces* in some convenient fashion. Usually, the division is performed so that the shape of each face has a compact mathematical representation, e.g., that the face lies on a single planar, quadratic, toroidal, or parametric surface. In order to guarantee that the subdivision corresponds with a legal plane model as discussed in Section 3.5, it is usually required to satisfy certain "topological" criteria to be discussed in the sequel.

The portion of the underlying surface that forms the face is "chalked out" in terms of a closed curve that lie on the surface. A face may well have several bounding curves, provided that they define a connected object. That is, faces are like "continents" that may have "lakes"; "isles" on lakes do not belong to faces, however.

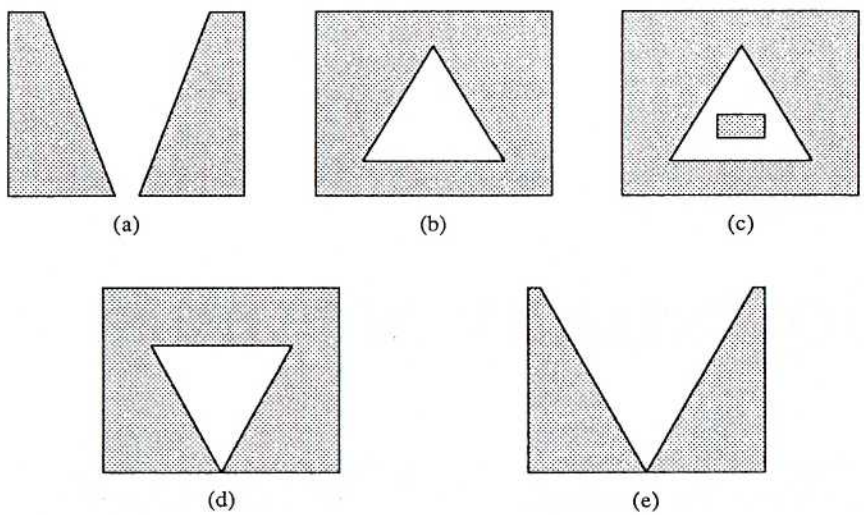



**Figure 6.1** Definition of a face.

The definition of a face is depicted in Figure 6.1. In the figure, cases (a) and (c) are disconnected 2-dimensional sets and would be represented as two faces. Cases (d) and (e) are somewhat exceptional faces whose boundaries "touch" themselves. Usually case (d) would be considered a good face, whereas (e) would be represented as two faces. Observe that the interior of (d) is connected, and of (e) not. Case (c) is a face with two boundaries.

In turn, the bounding curves of faces are represented through a division into *edges*. Analogously to the above, edges are chosen so as to have a convenient representation, say, a parametric equation. The portion of the curve that forms the edge is chalked out in terms of two *vertices*.

Figure 6.2 illustrates the basic components of a boundary model. In the figure, the surface of the object is divided into an enclosing set of faces (a), each of which is represented in terms of its bounding polygon (b), in turn represented in terms of edges and vertices (c).

## 6.2 BOUNDARY DATA STRUCTURES

The three object types *face*, *edge*, and *vertex*, and the geometric information attached to them form the basic constituents of boundary models. In



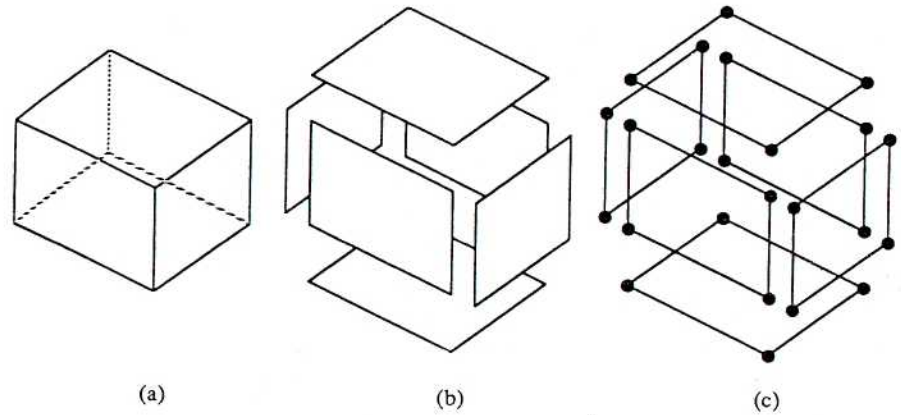



**Figure 6.2** Basic constituents of boundary models.

addition to geometric information such as face and curve equations and vertex coordinates, a boundary model must also represent how the faces, edges, and vertices are related to each other. It is customary to bundle all information of the geometry of the entities under the term *geometry* of a boundary model, and similarly information of their interconnections under the term *topology*.

All boundary models represent faces in terms of explicit nodes of a boundary data structure. After that, many alternatives for representing the geometry and the topology of a boundary model are possible, some of which are described below. For simplicity and clarity, we shall illustrate them in terms of alternative representations for the rectilinear block shown in Figure 6.3. Reference [8] discusses further alternatives and provides information on the data structures used in a number of modelers.

### 6.2.1 Polygon-Based Boundary Models

A boundary model that has only planar faces is called a *polyhedral model*. Because all edges of a polyhedron are straight line segments, it is possible to shrink the boundary data structure considerably in this important special case.

In the simplest variation faces are represented as *polygons*, each polygon consisting of a sequence of coordinate triples. A solid consists of a collection of faces grouped together in terms of, say, a table of face identifiers or a linked list of face nodes. Sometimes even the grouping information is elim-

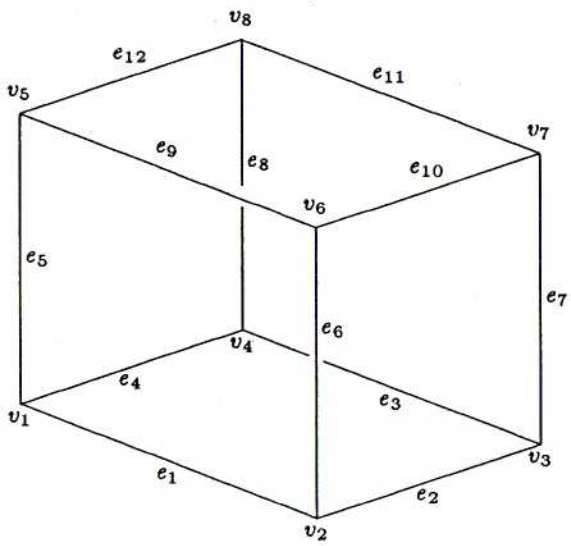



**Figure 6.3** A sample object.

inated and face relationships are completely implicit. This representation is usually used in *graphical metafiles* of graphics systems.

### 6.2.2 Vertex-Based Boundary Models

In a polygon-based boundary model vertex coordinates appear as often as the vertex appears in a face. This redundancy can be eliminated by introducing vertices as independent entities of the boundary data structure. In this case, vertex identifiers (or something equivalent, such as pointers to vertex nodes) are associated with faces. This approach leads us to various *vertex-based boundary models.*

Note that the sample representation of Figure 6.4 lists vertices of each face in a *consistent order*, namely clockwise as seen from the outside of the cube. This consistent orientation is useful in many algorithms. For instance, in hidden line or surface removal, it allows the elimination of *back faces* on the basis of face normal vectors pointing consistently away from the material. In the case of Figure 6.4, faces $f_1$, $f_4$, and $f_5$ would immediately be discarded by the hidden line or surface removal algorithm.

The representation of Figure 6.4 does not include any surface informa-



| *vertex* | *coordinates* |
|---|---|
| $v_1$ | $x_1\ y_1\ z_1$ |
| $v_2$ | $x_2\ y_2\ z_2$ |
| $v_3$ | $x_3\ y_3\ z_3$ |
| $v_4$ | $x_4\ y_4\ z_4$ |
| $v_5$ | $x_5\ y_5\ z_5$ |
| $v_6$ | $x_6\ y_6\ z_6$ |
| $v_7$ | $x_7\ y_7\ z_7$ |
| $v_8$ | $x_8\ y_8\ z_8$ |

| *face* | *vertices* |
|---|---|
| $f_1$ | $v_1\ v_2\ v_3\ v_4$ |
| $f_2$ | $v_6\ v_2\ v_1\ v_5$ |
| $f_3$ | $v_7\ v_3\ v_2\ v_6$ |
| $f_4$ | $v_8\ v_4\ v_3\ v_7$ |
| $f_5$ | $v_5\ v_1\ v_4\ v_8$ |
| $f_6$ | $v_8\ v_7\ v_6\ v_5$ |

**Figure 6.4** A vertex-based boundary model.

tion at all; as all faces are planar, their geometries are completely defined through the coordinates of their vertices. On the other hand, if face equations would be needed often for numerical calculations (say, for shading), they could be associated with faces.

In general, many choices as to what information is stored explicitly and what information is left implicit (i.e., to be computed as needed) must be made when implementing a boundary model. In Figure 6.4, the explicit inclusion of vertex coordinates implicitly gives us also face equations. (The opposite approach of storing face equations explicitly and leaving vertex coordinates implicit leads to a rather peculiar half-space model appropriate for convex objects only.)

If some redundant geometric information is stored (like the face equations in the example above), subtle numerical problems may arise. Suppose that due to small inaccuracies in floating point calculations, vertices of a face do not lie exactly on the plane defined by its face equation. Which information is considered "correct"?

### 6.2.3 Edge-Based Boundary Models

If curved surfaces are present in a boundary model, it becomes useful to include also edge nodes explicitly in the boundary data structure to be able to store information of intersection curves of faces. (Edge nodes are also useful for recording topological relationships as discussed in the next section.)

An *edge-based boundary model* represents a face boundary in terms of a closing sequence of edges, or *loop* for short. Vertices of the face are represented only through edges. This approach leads us to the model of Figure 6.5.

| *edge* | *vertices* |
|---|---|
| $e_1$ | $v_1\ v_2$ |
| $e_2$ | $v_2\ v_3$ |
| $e_3$ | $v_3\ v_4$ |
| $e_4$ | $v_4\ v_1$ |
| $e_5$ | $v_1\ v_5$ |
| $e_6$ | $v_2\ v_6$ |
| $e_7$ | $v_3\ v_7$ |
| $e_8$ | $v_4\ v_8$ |
| $e_9$ | $v_5\ v_6$ |
| $e_{10}$ | $v_6\ v_7$ |
| $e_{11}$ | $v_7\ v_8$ |
| $e_{12}$ | $v_8\ v_5$ |

| *vertex* | *coordinates* |
|---|---|
| $v_1$ | $x_1\ y_1\ z_1$ |
| $v_2$ | $x_2\ y_2\ z_2$ |
| $v_3$ | $x_3\ y_3\ z_3$ |
| $v_4$ | $x_4\ y_4\ z_4$ |
| $v_5$ | $x_5\ y_5\ z_5$ |
| $v_6$ | $x_6\ y_6\ z_6$ |
| $v_7$ | $x_7\ y_7\ z_7$ |
| $v_8$ | $x_8\ y_8\ z_8$ |

| *face* | *edges* |
|---|---|
| $f_1$ | $e_1\ e_2\ e_3\ e_4$ |
| $f_2$ | $e_9\ e_6\ e_1\ e_5$ |
| $f_3$ | $e_{10}\ e_7\ e_2\ e_6$ |
| $f_4$ | $e_{11}\ e_8\ e_3\ e_7$ |
| $f_5$ | $e_{12}\ e_5\ e_4\ e_8$ |
| $f_6$ | $e_{12}\ e_{11}\ e_{10}\ e_9$ |

**Figure 6.5** An edge-based model.

The data structure indicates an *orientation* for each edge; say, edge $e_1$ is considered to be (positively) oriented from vertex $v_1$ to vertex $v_2$. Again, faces are consistently oriented, i.e., their edges are listed clockwise as viewed from the outside of the block. Note that each edge occurs in exactly two faces, once in its positive orientation, and once in the opposite (negative) orientation.

**Winged-Edge Data Structure** The inclusion of explicit nodes for each of the basic object types (face, edge, and vertex) opens the door for elaborating an edge-based boundary model further. For instance, to aid algorithms such as hidden surface removal and shading, explicit face-face neighborhood information can be added to the data structure of Figure 6.5 by associating edges with the identifiers of the two faces they separate.

The so-called *winged-edge data structure*, first introduced by Baumgart [13,14] gives a primary example of such elaborated edge-based boundary models. It goes one step further by representing also the loop information in edge nodes.

Because each edge $e$ appears in exactly two faces, exactly two other edges $e'$ and $e''$ appear after $e$ in these faces. Moreover, because of the consistent orientation of faces, $e$ occurs exactly once in its positive orientation, and exactly once in the opposite orientation.

The winged-edge data structure takes advantage of these structural properties by associating the identifiers of the two "next" edges with an



| *edge* | *vstart* | *vend* | *ncw* | *nccw* |
|---|---|---|---|---|
| $e_1$ | $v_1$ | $v_2$ | $e_2$ | $e_5$ |
| $e_2$ | $v_2$ | $v_3$ | $e_3$ | $e_6$ |
| $e_3$ | $v_3$ | $v_4$ | $e_4$ | $e_7$ |
| $e_4$ | $v_4$ | $v_1$ | $e_1$ | $e_8$ |
| $e_5$ | $v_1$ | $v_5$ | $e_9$ | $e_4$ |
| $e_6$ | $v_2$ | $v_6$ | $e_{10}$ | $e_1$ |
| $e_7$ | $v_3$ | $v_7$ | $e_{11}$ | $e_2$ |
| $e_8$ | $v_4$ | $v_8$ | $e_{12}$ | $e_3$ |
| $e_9$ | $v_5$ | $v_6$ | $e_6$ | $e_{12}$ |
| $e_{10}$ | $v_6$ | $v_7$ | $e_7$ | $e_9$ |
| $e_{11}$ | $v_7$ | $v_8$ | $e_8$ | $e_{10}$ |
| $e_{12}$ | $v_8$ | $v_5$ | $e_5$ | $e_{11}$ |

| *vertex* | *coordinates* |
|---|---|
| $v_1$ | $x_1\ y_1\ z_1$ |
| $v_2$ | $x_2\ y_2\ z_2$ |
| $v_3$ | $x_3\ y_3\ z_3$ |
| $v_4$ | $x_4\ y_4\ z_4$ |
| $v_5$ | $x_5\ y_5\ z_5$ |
| $v_6$ | $x_6\ y_6\ z_6$ |
| $v_7$ | $x_7\ y_7\ z_7$ |
| $v_8$ | $x_8\ y_8\ z_8$ |

| *face* | *first edge* | *sign* |
|---|---|---|
| $f_1$ | $e_1$ | + |
| $f_2$ | $e_9$ | + |
| $f_3$ | $e_6$ | + |
| $f_4$ | $e_7$ | + |
| $f_5$ | $e_{12}$ | + |
| $f_6$ | $e_9$ | − |

**Figure 6.6** The winged-edge data structure.

edge node. By convention, these data are denoted by *ncw* and *nccw* for "next clockwise" and "next counterclockwise"; in particular, *ncw* identifies the next edge in the face where the edge occurs in its positive orientation, and *nccw* the next edge in the other face.

By virtue of this edge representation, faces only need to include the identifier of an arbitrary edge and a bit that indicates its orientation. Figure 6.6 gives an example of the winged-edge data structure. Signs + and − denote the orientations of the edge.

Starting from the edge directly associated with a loop, all other edges may be retrieved by following the *ncw* and *nccw* pointers. For instance, in the case of Figure 6.6, the boundary of face $f_5$ is extracted by starting from $e_{12}$. The next edge is given by $ncw(e_{12}) = e_5$. Because $e_5$ was traversed in its negative orientation (which can be seen by examining vertex identifiers), the next edge is now given by $nccw(e_5) = e_4$. Similarly we get

$nccw(e_4) = e_8$ as the next edge. It is traversed in its positive orientation, so the next edge is $ncw(e_8) = e_{12}$ again, and we know that all edges have been extracted.

In the most general variation, edge nodes of the winged-edge data structure also include the identifiers *fcw* and *fccw* of its neighbor faces, and analogously to *ncw* and *nccw*, the identifiers *pcw*, *pccw* of the previous edges in these faces. The orientation indicators of Figure 6.6 now become redundant. The resulting "full" winged-edge data structure is shown in Figure 6.7. The schematic diagram (a) indicates once more the meaning of the various data items.

Because the full winged-edge data structure includes the identifier of an adjacent edge into each vertex node, all edges meeting at a vertex can be extracted by an algorithm similar to the loop extraction algorithm (see Figure 6.7(a)).

In fact, the data structure treats faces and vertices in a completely symmetric fashion. If the nodes *vstart* and *vend*, and the nodes *fcw* and *fccw* are exchanged, we end up with the *dual* of the original polyhedron.

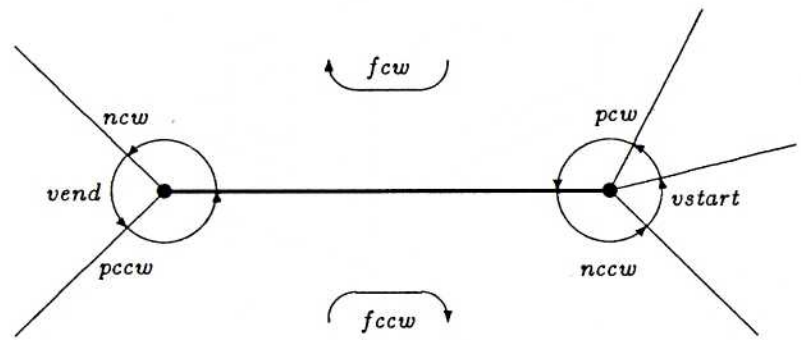
### 6.2.4 Faces With Several Boundaries

The data structures described so far assume that each face is *simply-connected*, i.e., that it has just one boundary curve. However, sometimes it seems natural to include faces that have several boundary curves; see, for instance, the object depicted in Figure 6.8.

Two alternatives present themselves. One approach is to model faces with nonsimple boundaries by connecting the boundary segments with auxiliary edges. (In [134], this representation was termed the "bridge edge representation".) In the case of Figure 6.8, this could take place by adding the dashed edge. Unfortunately, these special edges should be marked somehow to avoid their inclusion in line drawings. However, in the winged-edge data structure this is not necessary because auxiliary edges are considered to occur twice in the same face, and can be detected by comparing the *fcw* and *fccw* data items.

The other approach is to add a separate loop node that models a simple boundary into the boundary model, and associate each face with a list of loops. References to faces would be replaced by references to the appropriate loops; for instance, face identifiers *fcw* and *fccw* in edge nodes of the full winged-edge data structure would be replaced with the analogous loop identifiers *lcw* and *lccw*. All data structures discussed can be readily generalized in this fashion, and we will not elaborate on this topic further.






(a)

| *edge* | *vstart* | *vend* | *fcw* | *fccw* | *ncw* | *pcw* | *nccw* | *pccw* |
|---|---|---|---|---|---|---|---|---|
| $e_1$ | $v_1$ | $v_2$ | $f_1$ | $f_2$ | $e_2$ | $e_4$ | $e_5$ | $e_6$ |
| $e_2$ | $v_2$ | $v_3$ | $f_1$ | $f_3$ | $e_3$ | $e_1$ | $e_6$ | $e_7$ |
| $e_3$ | $v_3$ | $v_4$ | $f_1$ | $f_4$ | $e_4$ | $e_2$ | $e_7$ | $e_8$ |
| $e_4$ | $v_4$ | $v_1$ | $f_1$ | $f_5$ | $e_1$ | $e_3$ | $e_8$ | $e_5$ |
| $e_5$ | $v_1$ | $v_5$ | $f_2$ | $f_5$ | $e_9$ | $e_1$ | $e_4$ | $e_{12}$ |
| $e_6$ | $v_2$ | $v_6$ | $f_3$ | $f_2$ | $e_{10}$ | $e_2$ | $e_1$ | $e_9$ |
| $e_7$ | $v_3$ | $v_7$ | $f_4$ | $f_3$ | $e_{11}$ | $e_3$ | $e_2$ | $e_{10}$ |
| $e_8$ | $v_4$ | $v_8$ | $f_5$ | $f_4$ | $e_{12}$ | $e_4$ | $e_3$ | $e_{11}$ |
| $e_9$ | $v_5$ | $v_6$ | $f_2$ | $f_6$ | $e_6$ | $e_5$ | $e_{12}$ | $e_{10}$ |
| $e_{10}$ | $v_6$ | $v_7$ | $f_3$ | $f_6$ | $e_7$ | $e_6$ | $e_9$ | $e_{11}$ |
| $e_{11}$ | $v_7$ | $v_8$ | $f_4$ | $f_6$ | $e_8$ | $e_7$ | $e_{10}$ | $e_{12}$ |
| $e_{12}$ | $v_8$ | $v_5$ | $f_5$ | $f_6$ | $e_5$ | $e_8$ | $e_{11}$ | $e_9$ |

| *vertex* | *first edge* | *coordinates* |
|---|---|---|
| $v_1$ | $e_1$ | $x_1\ y_1\ z_1$ |
| $v_2$ | $e_2$ | $x_2\ y_2\ z_2$ |
| $v_3$ | $e_3$ | $x_3\ y_3\ z_3$ |
| $v_4$ | $e_4$ | $x_4\ y_4\ z_4$ |
| $v_5$ | $e_9$ | $x_5\ y_5\ z_5$ |
| $v_6$ | $e_{10}$ | $x_6\ y_6\ z_6$ |
| $v_7$ | $e_{11}$ | $x_7\ y_7\ z_7$ |
| $v_8$ | $e_{12}$ | $x_8\ y_8\ z_8$ |

| *face* | *first edge* |
|---|---|
| $f_1$ | $e_1$ |
| $f_2$ | $e_9$ |
| $f_3$ | $e_6$ |
| $f_4$ | $e_7$ |
| $f_5$ | $e_{12}$ |
| $f_6$ | $e_9$ |

(b)

**Figure 6.7** The full winged-edge data structure.

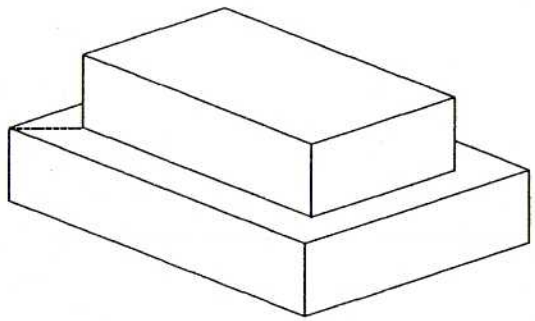


**Figure 6.8** Object with non-simple faces.

## 6.3 VALIDITY OF BOUNDARY MODELS

A boundary model is *valid* if it defines the boundary of a "reasonable" solid object. According to the theory of Section 3.5, we are usually only interested on objects bounded by closed, orientable surfaces. In that case, the validity criteria of a boundary model includes the following conditions:

1. The set of faces of the boundary model "closes," i.e., forms the complete "skin" of the solid with no missing parts.

2. Faces of the model do not intersect each other except at common vertices or edges.

3. The boundaries of faces are simple polygons that do not intersect themselves.

The first and second conditions exclude self-intersecting objects such as the object of Figure 6.9(a). Observe that the pyramid-shaped bottom surface of the object intersects the top surface. The third condition disallows objects such as the "open box" (b). (To represent "open" objects with a solid boundary model, the best approach is to model "openings" as specially marked faces, instead of leaving them out.)

The first condition relates to the *topological integrity* of a boundary model. The theory of plane models of Section 3.5 tells us that all topological integrity criteria can be enforced just by structural means. In particular, according to Definition 3.8, the first condition can be enforced by demanding that each edge occurs in exactly two faces; hence, no edge can



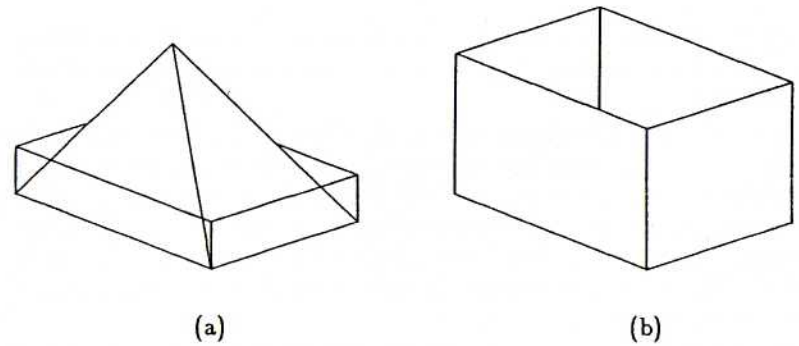



**Figure 6.9** Invalid boundary models.

be the boundary of a missing part of the surface. Observe that the winged-edge data structure "automatically" satisfies this criterion, because edges occurring in just one face cannot be represented.

The winged-edge data structure also enforces that the Möbius' rule of Definition 3.9 is satisfied; in fact, the faces will always be consistently oriented. Hence the surface represented by it is guaranteed to be orientable. However, nonorientable objects are disallowed also by the second condition above, because they would always intersect themselves in the three-dimensional space.

Unfortunately, the *geometric integrity* of a boundary model defined by the second and third conditions cannot be enforced just by structural means: by assigning inappropriate geometric information to a completely reasonable topological entities, invalid models can be created. To guarantee the geometric integrity, one must either resort to a computationally expensive test that involves a comparison of each pair of faces in the solid, or limit the user's freedom by giving him only validity-enforcing solid description mechanisms.

## 6.4 DESCRIPTION OF BOUNDARY MODELS

Examination of Figures 6.6 and 6.7 soon reveals one of the major problems of boundary models, namely the complexity of their construction. It is beyond the capabilities of a normal human being to build correct boundary models directly, e.g., by typing information such as that of Figure 6.7.

The designer of a boundary modeler is therefore faced with the need of providing a sufficient collection of more convenient and efficient solid de-

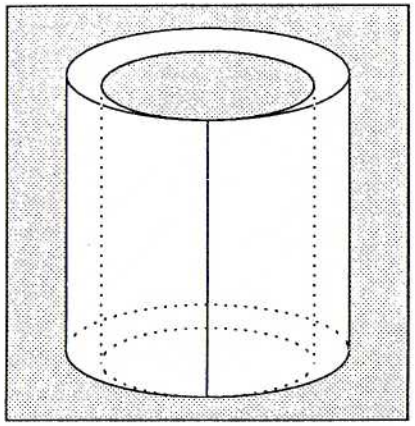


**Figure 6.10** An object with no convenient BR.

scription facilities. On the other hand, he must take the integrity problems of boundary models discussed in the previous section into account, and seek for solid description techniques that either guarantee the integrity criteria, or (al least) make it difficult to construct invalid models.

Some frequently used description mechanisms are discussed briefly below.

### 6.4.1 Conversion from CSG

A common solution is to give a CSG-based solid description language to the user and construct boundary models through conversion from CSG. In this approach, the boundary modeler must include algorithms for creating boundary models from the CSG primitives, and an algorithm for Boolean set operations on boundary models.

Unfortunately, set operations for boundary models are computationally expensive and sensitive for numerical problems. Even if the numerical problems of calculating surface intersections can be solved, the problem of the mismatch between the modeling spaces of CSG and conventional boundary models remains. That is, some objects that can be represented in CSG have no convenient representations as boundary models.

An example of this is shown in Figure 6.10. The object can be represented in CSG as the difference of two cylinders. On the other hand, ordinary boundary data structures (such as the winged-edge data structure)



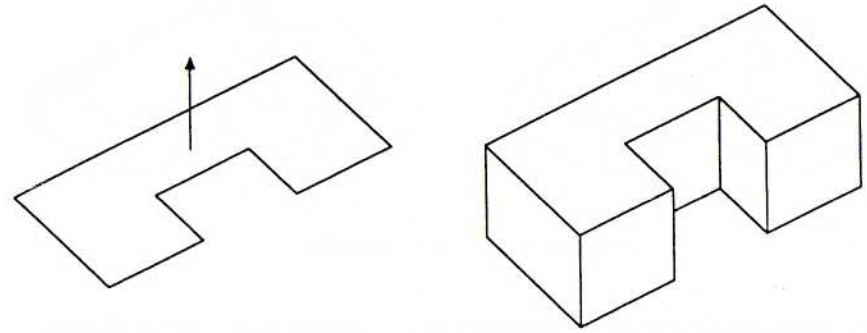


**Figure 6.11** Solid description by drawing and sweeping.

cannot represent properly the region where the "inner" cylinder touches the "outer" one. Because an edge cannot have four neighbor faces, a pair of coinciding edges must be used. This, however, breaks down the geometric integrity criteria, and problems may arise if additional Boolean operations are performed on the boudary model.

Reference [96] outlines a number of set operations algorithms for boundary models; we shall study the problems of set operations further in Chapter 15.

### 6.4.2 Two-and-Half-Dimensional Drawing

Many objects needed in practice exhibit symmetry that can be exploited in their description. Quite often the object can be described in terms of a two-dimensional cross section, and information of material "thickness." Rotationally symmetric objects give another example of how the solid can be described just by a profile and a rotation axis.

Sometimes both of these types of objects could be described in terms of basic objects and Boolean set operations. Nevertheless, many people find it far more natural to "draw" an outline, and "sweep" it to give it a thickness or "swing" it to make it a rotational solid. Because of the relatively large computations needed for evaluating Boolean operations of boundary models, these operations can be executed much more efficiently than the sequence of set operations that creates the same model.

These *two-and-half-dimensional* solid description mechanisms can naturally be incorporated to boundary modelers. The term "two-and-half-dimensional" emphasizes that most description operations take place in

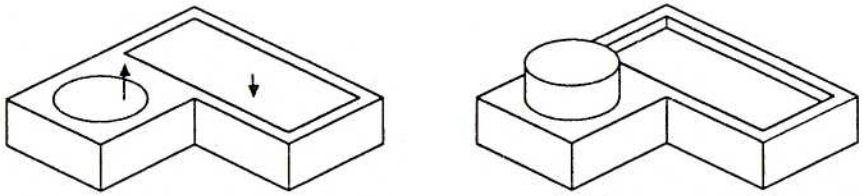


**Figure 6.12** Face sweeping.

two dimensions, and can be visualized as operations on a two-dimensional graphical model.

One possibility is to include separate sweeping primitives in the boundary modeler, and to provide a conversion algorithm from the sweeping model to a boundary model. This approach is completely analogous to the inclusion of sweeping primitives to CSG models described in Section 5.2.

A more general approach is to consider sweeping an operation that can be performed on *any* planar face of a boundary model. This "face lifting" operation allows, for instance, the construction sequence of the object depicted in Figure 6.12. This efficient and economical input style is heavily used in boundary modelers belonging to the so-called BUILD group of solid modelers [53].

Engineering drawings generally display an object from several orthogonal views. This technique can be used for describing boundary models in the form of the so-called *profile set operations*. This operation combines two (or several) closed profiles, representing the outlines of an object from two views, by sweeping the outlines by an appropriate amount and evaluating the Boolean intersection of the swept objects. Surprisingly many objects can be modeled quite satisfactorily with this operation only; see, for instance, Figure 6.13.

Unfortunately, the sweeping operations are not completely "safe" either; is is easy to create self-intersecting objects with them, for example. See [18] for discussion on conditions under which sweeping and similar operations can be kept integrity-preserving.

### 6.4.3 Local Modification

A third natural solid description facility is based on *local modifications* of a boundary model. Actually, even the face lifting operation of the preceding section can be understood as a local modification that replaces a face by a



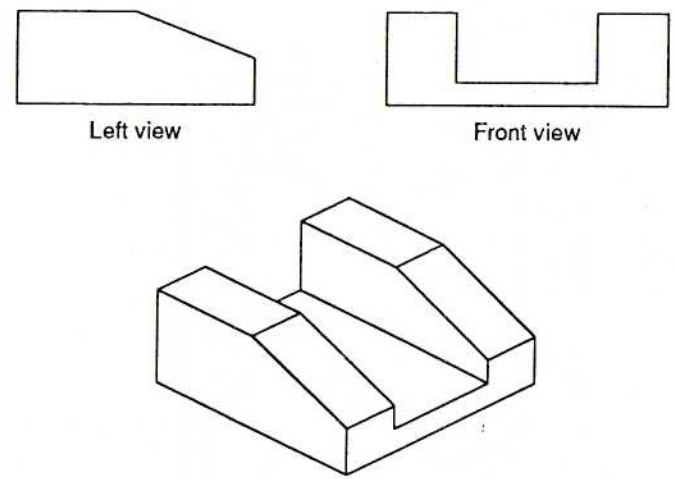



**Figure 6.13** A sample profile set operation.

collection of new faces modeling the lifted region.

The boundary modelers BUILD-2 [18] and ROMULUS include a very rich collection of local modification operations. A typical example is the "rounding" of a sharp edge as depicted in Figure 6.14(a). A more general case of this is the "blending" of several adjacent edges depicted in (b); note how a vertex is replaced with a triangular blend surface.

The boundary modeler MODIF [25] includes another kind of a local modification operation. In MODIF, a solid is first modeled as a polyhedron. The polyhedron is then converted into a curved surface model by introducing control points on each edge. The user is then allowed to modify the coordinates of control points in order to create the desired shape. Special techniques are employed to keep the surface smooth.

## 6.5 ALGORITHMS FOR BOUNDARY MODELS

### 6.5.1 Visualization

The design of visualization algorithms for boundary models is greatly simplified by the availability of the faces, edges, and vertices of a solid. In this respect, they may be considered "explicit" models in comparison to CSG models that must be "evaluated" for visualization purposes.

All techniques for generating graphical output of graphical models are directly applicable to boundary models. In addition, for a polyhedral model

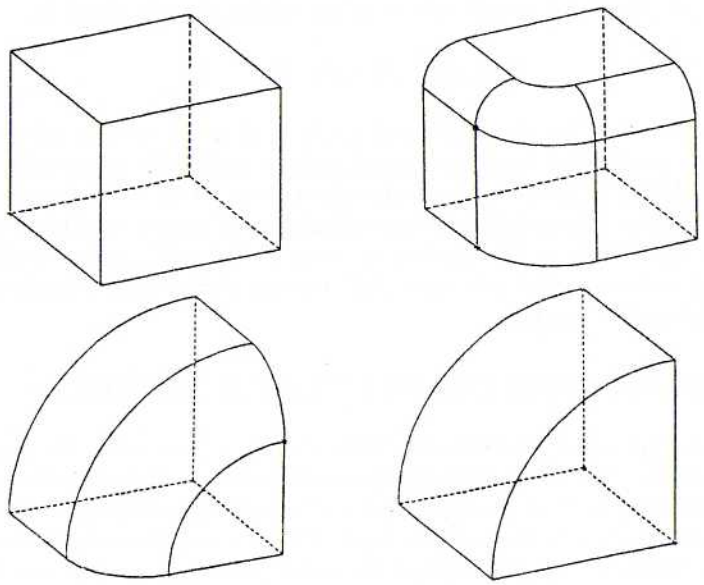


**Figure 6.14** Local modifications of boundary models.

all well-known techniques of hidden line and surface removal and shading can readily be applied, including "exact" object-space methods. Furthermore, advanced graphical display devices often include hardware support for the rapid processing of polyhedral models.

Unfortunately, the presence of curved surfaces makes things far more complicated especially if hidden line output is required. There are specialized hidden line removal algorithms that can operate also on natural quadratic surfaces (see [70]), but in general it is more efficient to convert curved-surface models into polyhedral ones in order to create hidden line output.

Hidden surface removal is (surprisingly) simpler, because standard techniques such as *scan-line algorithms*, the *z-buffer algorithm*, and ray casting are applicable. These methods allow the generation of very high quality images at medium to high computational cost.

### 6.5.2 Integral Properties

Two general techniques are available for the calculation of basic engineering properties based on boundary models, namely the method of *direct integration* and the use of the *divergence theorem* of calculus [69].



Direct integration is the standard technique discussed in calculus textbooks. This technique is based on evaluating a volume integral over a solid as the sum of the appropriately signed contributions of its faces; see Figure 6.15(a).

For instance, the volume integral of a function $f(x, y, z)$ over a solid $S$ can be evaluated by

$$\int\int\int_S f(x,y,z)dzdydx = \pm\sum_i \int\int_{F_i'} \left(\int_0^{z_i(x,y)} f(x,y,z)dz\right) dydx \tag{6.1}$$

where $F_i'$ is the projection of face $F_i$ on $xy$-plane, and $z_i(x, y)$ is obtained by solving the equation of $F_i$ for $z$. The sign is determined by looking at the surface normal of the face: if the normal is facing the $xy$-plane, the minus is selected, otherwise the plus.

The resulting double integral can be evaluated by applying a similar technique to evaluating the contribution of each edge of $F_i$; see Figure 6.15(b). That is, the area integral of a function $g(x, y)$ over the face can be evaluated by

$$\int\int_{F_i'} g(x,y)dydx = \pm\sum_j \int_{E_j'} \left(\int_0^{y_j(x)} g(x,y)dy\right) dx \tag{6.2}$$

where $E_j'$ is the projection of $E_j$ on $x$-axis, and $y_j(x)$ the equation of $E_j$ solved for $y$. This will finally lead us to single integrals of the form

$$\int_{x_{j,1}}^{x_{j,0}} h(x)dx. \tag{6.3}$$

where $x_{j,0}$ and $x_{j,1}$ are the (projected) endpoints of $E_j$.

Observe that both $z_i(x, y)$ and $y_j(x)$ are assumed to be single-valued in $x$ and $y$. Hence curved faces and edges may have to be subdivided. It may still be hard to solve the surface and curve equations for $z$ and $y$ even numerically, and the equations of the projected edges $E_j'$ of $F_i'$ may not even be available. However, for polyhedral models the direct integration is quite attractive, because face equations are either single-valued or can be ignored, and straight lines project to straight lines.

The divergence theorem provides an alternative method for the evaluation of integral properties [122]. From the observation that it is always possible to find at least one vector function $g(x, y, z)$ such that $\mathrm{div}g = f$ for any given continuous function $f(x, y, z)$, it follows that

$$\int_S f\, dV = \int_S \mathrm{div}g\, dV = \sum_i \int_{F_i} g \cdot n_i\, dF_i$$

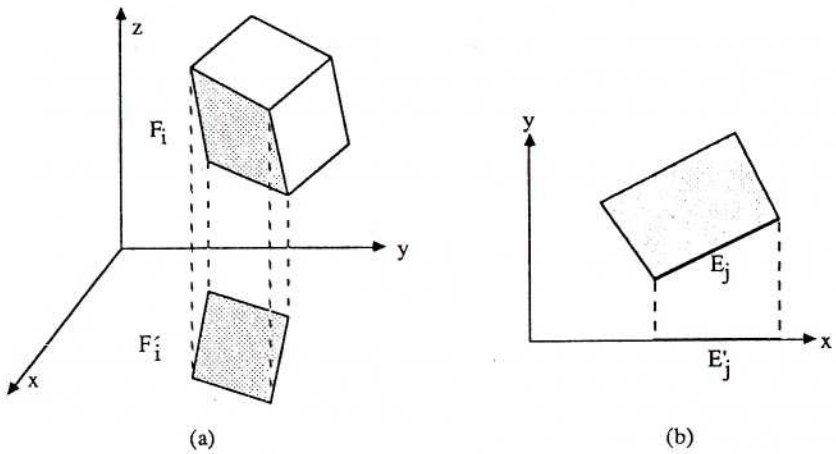



**Figure 6.15** The method of direct integration.

where $F_i$ is a face of the solid, $S$, $n_i$ is the unit normal vector to $F_i$, and $dF_i$ is the surface differential. For instance, if

$$f(x, y, z) = y^2 + z^2,$$

then, for example, the choices

$$g(x, y, z) = [x(y^2 + z^2) \;\; 0 \;\; 0]$$

and

$$g(x, y, z) = [0 \;\; y^3/3 \;\; z^3/3]$$

are both possible. If all $F_i$ are planar polygons, integrals on the right side can be readily evaluated. In the general case, approximation techniques are needed; see [122] for additional information and practical results.

In addition to the general methods discussed above, very straightforward direct techniques for evaluating integral properties of consistently oriented polyhedral objects are available. For instance, the volume of a polyhedron can be reduced to calculating to the sum of the signed volumes of tetrahedra [129].

Assume the tetrahedron has one corner at origin, and the other corners at

$$P_i = [x_i \; y_i \; z_i],$$



for $i = 1, 2, 3$. Then the signed volume of the tetrahedron is given by

$$V_{tetra} = \frac{1}{6} \; P_1 \times P_2 \cdot P_3$$

where $\times$ and  stand for the vector and scalar product of vectors. Observe that $V_{tetra}$ is positive or negative depending on whether the orientation of the triangle $P_1 \; P_2 \; P_3$ as viewed from the origin is clockwise or counterclockwise. The volume of a polyhedron can calculated by taking all tetrahedra formed by three consecutive vertices in faces of the solid, and summing their signed volumes. The reference [129] reports very similar algorithms for other integral properties.

## 6.6 PROPERTIES OF BOUNDARY MODELS

We can again formulate the summary of this chapter by discussing the properties of boundarv models within the general framework of Section 3.6.1.

*Expressive power*: The modeling space of boundary models depends on the selection of surfaces that can be used. There is no inherent reason to limit this collection to mere half-spaces; hence boundary models can be used to represent objects from a more general modeling space than that possible for CSG.

On the other hand, as noted in Section 6.4.1, some objects that can be represented in CSG have no convenient representations as boundary models. While knowledgeable users can avoid such objects when working in direct interaction with a boundary modeler, this deficiency may cause problems e.g. if conversion from a CSG model is attempted.

*Validity*: Validity of boundary models is in general quite difficult to establish. Validity criteria split into topological constraints and geometric constraints as discussed above. While it is possible to manage topological validity without large overhead, it is hard to enforce geometric correctness without penalizing the speed in interactive design.

*Unambiguity and uniqueness*: Valid boundary models are unambiguous. They are not unique.

*Description languages*: Boundary models are tedious to describe directly. Fortunately, it is possible to build description languages that are based on graphical "drawing" and "sweeping" operations or on a CSG-like input on top of a boundary model.

*Conciseness*: Boundary models of useful objects may become large, especially if curved objects are approximated with polyhedral models.

*Closure of operations*: As discussed in Chapter 3, boundary models are usually *not* closed under set operations (regularized or not). In this theoretical sense boundary model description mechanisms based on CSG

conversion or set operations are always vulnerable. The natural closed operations for boundary models are the *Euler operators* to be studied in Chapter 9.

*Computational ease and applicability*: Boundary models are useful for generating graphical output, because they readily include the data needed for driving a graphical display. Analysis algorithms based directly on boundary models become quite difficult if representations of objects from more general modeling spaces than polyhedra must be processed.

We shall return to boundary models in much more detail in Part Two of this book.

## PROBLEMS

6.1. Describe an algorithm for extracting the edges around a vertex from the full winged-edge data structure of Figure 6.6.

6.2. Form the dual of the full winged-edge data structure of Figure 6.6 as indicated in the last paragraph of Section 6.2.3. Which object do you get?

6.3. Present (in the style of Figure 6.6) a modified winged-edge data structure of the "pyramid" of Figure 6.7 that can represent faces with several boundaries.

6.4. Based on the general Equations 6.1–3, outline an algorithm for evaluating the following integral properties of a polyhedral solid $S$:

(a) the volume $V$, which is expressed as the triple integral

$$V = \int\int\int_S dzdydx$$

(b) the moment of inertia about $z$,

$$I_z = \int\int\int_S x^2 + y^2 \, dzdydx$$

## BIBLIOGRAPHIC NOTES

An overview of boundary data structures and the representations used in a number of modelers is given in the survey by Baer, Eastman, and Henrion [8], which still is recommended reading for a student of solid modeling.

The solid modelers BUILD-1 and BUILD-2 written by the former BUILD-group at the University of Cambridge under the leadership of Ian Braid belong to the most influential works to the solid modeling technology. The report [18] provides an overview of the design and implementation of the version of BUILD-2 of that



time, and is a must to anybody contemplating the design of a boundary modeler. Work on BUILD modelers still continues at the Cranfield Institute for Technology.

A good reference to scan-line techniques is [102].

Section 6.5.2 is based on the survey article on integral properties by Lee and Requicha [69].